

Raspberry Pi Steps for Apex Rover

Dual Camera Server + Sensor Bridge

Goal

Run the Raspberry Pi so it does two jobs:

1. Stream two cameras to the mobile app:
2. Front Camera for the robot
3. Arm Camera for the robot arm
4. Forward sensor data:
5. Arduino Mega sends sensor data to Raspberry Pi
6. Raspberry Pi sends the data to ESP32
7. ESP32 broadcasts the data to the mobile app using WebSocket

Important:

The Raspberry Pi does **not** control robot movement.

All movement control is manual only from the mobile app:

```
Mobile App → ESP32 → Mega / UNO
```

The Raspberry Pi is only responsible for:

```
Camera Stream + Sensor Bridge
```

1. Enter the Apex Rover Project

Run this on the Raspberry Pi:

```
cd ~/ApexRover_Repo
```

```
pwd  
  
git status  
  
git pull origin main  
  
ls
```

Note:

The `ls` command is only used to make sure the project files exist after updating.

2. Connect to the ESP32 Network

The Raspberry Pi must be connected to the robot network:

```
Apex_Rover_Net
```

If it is not connected, run:

```
sudo nmcli dev wifi connect "Apex_Rover_Net" password "12345678"
```

Then check the Raspberry Pi IP address:

```
hostname -I
```

It should show something like:

```
192.168.4.2
```

Then test the connection with the ESP32:

```
ping -c 4 192.168.4.1
```

Then run:

```
curl http://192.168.4.1/get_status
```

It should return something close to:

```
{"ok": true, "mode": "MANUAL"}
```

Or it should show that the ESP32 is running in Manual Only mode.

3. Install Camera and Server Libraries

Run:

```
sudo apt update  
  
sudo apt install python3-flask python3-opencv v4l-utils fswebcam -y
```

4. Check the Cameras

Run:

```
ls -l /dev/v4l/by-id/
```

It should show something similar to:

```
usb-046d_HD_Pro_Webcam_C920_A59D7FAF-video-index0  
usb-046d_HD_Pro_Webcam_C920_A59D7FAF-video-index1  
usb-GENERAL_2K_HD_Camera-video-index0  
usb-GENERAL_2K_HD_Camera-video-index1
```

Based on the previous check:

```
Front Camera = usb-GENERAL_2K_HD_Camera-video-index0  
Arm Camera   = usb-046d_HD_Pro_Webcam_C920_A59D7FAF-video-index0
```

5. Create the Dual Camera Server File

Run:

```
cd ~/ApexRover_Repo

cat > dual_camera_server.py <<'PY'
import cv2
import time
from flask import Flask, Response, jsonify

# =====
# Apex Rover Dual Camera Server
#
# Raspberry Pi responsibility:
#   1) Front camera stream
#   2) Arm camera stream
#
# Raspberry Pi DOES NOT control robot movement.
#
# Mobile App URLs:
#   http://192.168.4.2:5000/front_camera
#   http://192.168.4.2:5000/arm_camera
#   http://192.168.4.2:5000/front_snapshot
#   http://192.168.4.2:5000/arm_snapshot
#   http://192.168.4.2:5000/status
# =====

FRONT_CAMERA_DEVICE = "/dev/v4l/by-id/usb-GENERAL_2K_HD_Camera-video-index0"
ARM_CAMERA_DEVICE = "/dev/v4l/by-id/usb-046d_HD_Pro_Webcam_C920_A59D7FAF-video-index0"

FRAME_WIDTH = 640
FRAME_HEIGHT = 480
FRAME_FPS = 20
JPEG_QUALITY = 70

app = Flask(__name__)

class CameraStream:
    def __init__(self, name, device):
        self.name = name
        self.device = device
        self.cap = None
```

```

def open(self):
    if self.cap is not None:
        self.cap.release()
        self.cap = None

    print(f"[INFO] Opening {self.name}: {self.device}")

    self.cap = cv2.VideoCapture(self.device, cv2.CAP_V4L2)
    self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, FRAME_WIDTH)
    self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, FRAME_HEIGHT)
    self.cap.set(cv2.CAP_PROP_FPS, FRAME_FPS)

    if self.cap.isOpened():
        print(f"[OK] {self.name} opened")
        return True

    print(f"[ERROR] Could not open {self.name}")
    return False

def is_opened(self):
    return self.cap is not None and self.cap.isOpened()

def read_frame(self):
    if not self.is_opened():
        self.open()

    if not self.is_opened():
        return None

    ret, frame = self.cap.read()

    if not ret or frame is None:
        print(f"[WARN] Failed to read from {self.name}, reopening...")
        self.open()
        return None

    return frame

def encode_jpeg(self, frame):
    ok, buffer = cv2.imencode(
        ".jpg",
        frame,
        [int(cv2.IMWRITE_JPEG_QUALITY), JPEG_QUALITY],
    )

    if not ok:
        return None

```

```

        return buffer.tobytes()

    def generate_mjpeg(self):
        while True:
            frame = self.read_frame()

            if frame is None:
                time.sleep(0.1)
                continue

            jpg = self.encode_jpeg(frame)

            if jpg is None:
                continue

            yield (
                b"--frame\r\n"
                b"Content-Type: image/jpeg\r\n\r\n" + jpg + b"\r\n"
            )

            time.sleep(0.03)

    def snapshot(self):
        frame = self.read_frame()

        if frame is None:
            return None

        return self.encode_jpeg(frame)

front_camera = CameraStream("Front Camera", FRONT_CAMERA_DEVICE)
arm_camera = CameraStream("Arm Camera", ARM_CAMERA_DEVICE)

@app.route("/")
def home():
    return """
Apex Rover Dual Camera Server Running

Front Camera:
    /front_camera
    /front_snapshot

Arm Camera:
    /arm_camera
    /arm_snapshot

```

```

Status:
    /status
"""

@app.route("/status")
def status():
    return jsonify({
        "ok": True,
        "front_camera_device": FRONT_CAMERA_DEVICE,
        "arm_camera_device": ARM_CAMERA_DEVICE,
        "front_opened": front_camera.is_opened(),
        "arm_opened": arm_camera.is_opened(),
        "front_stream": "/front_camera",
        "arm_stream": "/arm_camera",
        "front_snapshot": "/front_snapshot",
        "arm_snapshot": "/arm_snapshot",
    })

@app.route("/front_camera")
def front_camera_stream():
    return Response(
        front_camera.generate_mjpeg(),
        mimetype="multipart/x-mixed-replace; boundary=frame",
    )

@app.route("/arm_camera")
def arm_camera_stream():
    return Response(
        arm_camera.generate_mjpeg(),
        mimetype="multipart/x-mixed-replace; boundary=frame",
    )

@app.route("/front_snapshot")
def front_snapshot():
    jpg = front_camera.snapshot()

    if jpg is None:
        return "Front camera not available", 503

    return Response(jpg, mimetype="image/jpeg")

@app.route("/arm_snapshot")
def arm_snapshot():

```

```
    jpg = arm_camera.snapshot()

    if jpg is None:
        return "Arm camera not available", 503

    return Response(jpg, mimetype="image/jpeg")

if __name__ == "__main__":
    front_camera.open()
    arm_camera.open()

    print("=====")
    print("Apex Rover Dual Camera Server")
    print("Front stream   : http://192.168.4.2:5000/front_camera")
    print("Arm stream      : http://192.168.4.2:5000/arm_camera")
    print("Front snapshot  : http://192.168.4.2:5000/front_snapshot")
    print("Arm snapshot    : http://192.168.4.2:5000/arm_snapshot")
    print("Status          : http://192.168.4.2:5000/status")
    print("=====")

    app.run(host="0.0.0.0", port=5000, threaded=True)
PY
```

6. Run the Camera Server

Run:

```
cd ~/ApexRover_Repo

python3 dual_camera_server.py
```

You should see something like:

```
[OK] Front Camera opened
[OK] Arm Camera opened
Apex Rover Dual Camera Server
```

If an error appears for any camera, send the error message.

7. Test the Server from the Raspberry Pi

Open another terminal and run:

```
curl http://127.0.0.1:5000/status
```

Then save a snapshot from the front camera:

```
curl http://127.0.0.1:5000/front_snapshot --output front_test.jpg
```

Then test the arm camera:

```
curl http://127.0.0.1:5000/arm_snapshot --output arm_test.jpg
```

Then run:

```
ls *.jpg
```

You should see:

```
front_test.jpg  
arm_test.jpg
```

8. Test the Camera from the Mobile Phone

The mobile phone must be connected to the same robot network:

```
Apex_Rover_Net
```

From the mobile browser, open:

```
http://192.168.4.2:5000/status
```

Then:

```
http://192.168.4.2:5000/front_snapshot
```

Then:

```
http://192.168.4.2:5000/arm_snapshot
```

If the images appear in the browser, the mobile app will be able to display them.

9. Run the Mobile App

If the project is on Windows:

```
cd C:\RobotGithup\Apex-Rover  
  
git pull origin main  
  
flutter pub get  
  
flutter run
```

If the project is on Linux, enter your Flutter project path, for example:

```
cd ~/VS_StormProjects/ApexRover/robot_master_control  
  
git pull origin main  
  
flutter pub get  
  
flutter run
```

From the app:

```
Connect  
Control  
Remote Control
```

Expected result:

Basic / Rear Jack / Front Jack should display the Front Camera.

Arm Mode should display the Arm Camera.

10. If the Raspberry Pi IP Changes

If this command:

```
hostname -I
```

shows an IP different from:

```
192.168.4.2
```

For example:

```
192.168.4.5
```

Then you must update the Flutter file:

```
lib/screens/remote_control_screen.dart
```

Change this line:

```
static const String raspberryBaseUrl = 'http://192.168.4.2:5000';
```

To:

```
static const String raspberryBaseUrl = 'http://192.168.4.5:5000';
```

11. Add Sensor Data Streaming to the Mobile App

After running the camera stream, we also need to run the sensor bridge.

The video path is:

```
Raspberry Pi → Mobile App
```

The sensor data path is:

```
Arduino Mega → Raspberry Pi → ESP32 → Mobile App
```

The mobile app expects a WebSocket message from the ESP32 in this format:

```
SENSOR:PITCH=2.4;ROLL=-1.1;FRONT=35.6;REAR=18.2;BALANCE=STABLE
```

Or:

```
SENSOR:PITCH=16.2;ROLL=4.1;FRONT=12.0;REAR=8.5;BALANCE=WARNING
```

Or:

```
SENSOR:PITCH=31.5;ROLL=27.2;FRONT=4.0;REAR=5.5;BALANCE=DANGER
```

So the Raspberry Pi must read the sensors from the Mega and send them to the ESP32 using this endpoint:

```
http://192.168.4.1/sensor_update
```

Very important:

In the final plan, the Mega automatically sends sensor data like this:

```
SENSOR:PITCH=2.30;ROLL=-1.10;UF=8.50;UR=9.20;ALERT=NONE
```

So `sensor_bridge.py` must read `SENSOR:` directly.

Do not use:

```
GET:SENSORS
```

Do not wait for:

DATA:

12. Install Sensor Libraries on Raspberry Pi

Run:

```
sudo apt update  
  
sudo apt install python3-serial python3-requests -y
```

13. Find the Arduino Mega Port on Raspberry Pi

Connect the Arduino Mega USB cable to the Raspberry Pi.

Then run:

```
ls /dev/ttyACM*
```

If it shows:

```
/dev/ttyACM0
```

Then keep this inside the code:

```
MEGA_PORT = "/dev/ttyACM0"
```

If it shows:

```
/dev/ttyACM1
```

Then use:

```
MEGA_PORT = "/dev/ttyACM1"
```

If nothing appears, try:

```
ls /dev/ttyUSB*
```

The port may be:

```
/dev/ttyUSB0
```

14. Create `sensor_bridge.py`

Run:

```
cd ~/ApexRover_Repo

cat > sensor_bridge.py <<'PY'
import time
import serial
import requests

# =====
# Apex Rover Sensor Bridge
#
# Flow:
#   Arduino Mega -> Raspberry Pi -> ESP32 -> Mobile App
#
# Mega sends automatically:
#   SENSOR:PITCH=2.30;ROLL=-1.10;UF=8.50;UR=9.20;ALERT=NONE
#
# Raspberry sends to ESP32:
#   http://192.168.4.1/sensor_update?
pitch=...&roll=...&front=...&rear=...&balance=...
#
# Raspberry Pi DOES NOT control robot movement.
# =====

ESP32_URL = "http://192.168.4.1"

# Change this if your port is /dev/ttyACM1 or /dev/ttyUSB0
MEGA_PORT = "/dev/ttyACM0"

BAUD_RATE = 9600
SERIAL_TIMEOUT = 0.5
```

```

FORWARD_TIMEOUT = 0.5

def compute_balance_status(pitch, roll, alert):
    abs_pitch = abs(pitch)
    abs_roll = abs(roll)

    if alert == "TILT_DANGER":
        return "DANGER"

    if alert in ("OBSTACLE_FRONT", "OBSTACLE_REAR"):
        return "WARNING"

    if abs_pitch >= 30 or abs_roll >= 25:
        return "DANGER"

    if abs_pitch >= 15 or abs_roll >= 12:
        return "WARNING"

    return "STABLE"

def parse_sensor_line(line):
    line = line.strip()

    if not line.startswith("SENSOR:"):
        return None

    payload = line[len("SENSOR:"):]
    parts = payload.split(";")

    data = {}

    for part in parts:
        if "=" not in part:
            continue

        key, value = part.split("=", 1)
        key = key.strip().upper()
        value = value.strip()
        data[key] = value

    try:
        pitch = float(data.get("PITCH", 0.0))
        roll = float(data.get("ROLL", 0.0))

        # Mega may send UF/UR or FRONT/REAR
        front = float(data.get("UF", data.get("FRONT", -1.0)))

```

```

        rear = float(data.get("UR", data.get("REAR", -1.0)))

        alert = data.get("ALERT", "NONE").upper()

        balance = compute_balance_status(pitch, roll, alert)

    return {
        "pitch": pitch,
        "roll": roll,
        "front": front,
        "rear": rear,
        "balance": balance,
        "alert": alert,
    }

except ValueError:
    print("[WARN] Bad SENSOR values:", line)
    return None

def send_to_esp32(sensor_data):
    try:
        response = requests.get(
            f"{ESP32_URL}/sensor_update",
            params={
                "pitch": f"{sensor_data['pitch']:.2f}",
                "roll": f"{sensor_data['roll']:.2f}",
                "front": f"{sensor_data['front']:.2f}",
                "rear": f"{sensor_data['rear']:.2f}",
                "balance": sensor_data["balance"],
            },
            timeout=FORWARD_TIMEOUT,
        )

        print("[ESP32]", response.text)

    except requests.RequestException as e:
        print("[ERROR] Could not send to ESP32:", e)

def main():
    print("=====")
    print("Apex Rover Sensor Bridge")
    print(f"Mega Port : {MEGA_PORT}")
    print(f"ESP32 URL : {ESP32_URL}")
    print("=====")

    while True:

```



```

        try:
            with serial.Serial(MEGA_PORT, BAUD_RATE, timeout=SERIAL_TIMEOUT) as
mega:
                time.sleep(2)
                mega.reset_input_buffer()

                print("[OK] Connected to Mega")
                print("[INFO] Waiting for SENSOR lines from Mega...")

                while True:
                    raw = mega.readline()

                    if not raw:
                        continue

                    line = raw.decode(errors="ignore").strip()

                    if line:
                        print("[MEGA]", line)

                        sensor_data = parse_sensor_line(line)

                        if sensor_data is not None:
                            print("[SENSOR]", sensor_data)
                            send_to_esp32(sensor_data)

            except serial.SerialException as e:
                print("[ERROR] Mega serial problem:", e)
                print("[INFO] Retrying in 2 seconds...")
                time.sleep(2)

if __name__ == "__main__":
    main()
PY

```

15. Run the Sensor Bridge

Open a new terminal on the Raspberry Pi and run:

```

cd ~/ApexRover_Repo

python3 sensor_bridge.py

```

You should see something like:

```
[OK] Connected to Mega
[INFO] Waiting for SENSOR lines from Mega...
[MEGA] SENSOR:PITCH=2.30;ROLL=-1.10;UF=8.50;UR=9.20;ALERT=NONE
[SENSOR] {'pitch': 2.3, 'roll': -1.1, 'front': 8.5, 'rear': 9.2, 'balance':
'STABLE', 'alert': 'NONE'}
[ESP32] {"ok":true,...
```

If it stays stuck on:

```
[INFO] Waiting for SENSOR lines from Mega...
```

That means one of these is wrong:

- Mega is not sending `SENSOR:` lines
- The serial port is wrong
- The Mega code was not uploaded correctly

16. Test Sensors Without the Mega

After uploading the new ESP32 code, run this from any device connected to `Apex_Rover_Net`:

```
curl "http://192.168.4.1/sensor_update?
pitch=2.4&roll=-1.1&front=35.6&rear=18.2&balance=STABLE"
```

Then open the mobile app Remote Control screen.

It should show:

```
Balance = STABLE
Pitch = 2.4°
Roll = -1.1°
Front US = 35.6 cm
Rear US = 18.2 cm
```

Test WARNING:

```
curl "http://192.168.4.1/sensor_update?  
pitch=16.2&roll=4.1&front=12.0&rear=8.5&balance=WARNING"
```

Test DANGER:

```
curl "http://192.168.4.1/sensor_update?  
pitch=31.5&roll=27.2&front=4.0&rear=5.5&balance=DANGER"
```

In WARNING or DANGER mode, the app should show an alert.

17. Run the Full System

Two programs must be running on the Raspberry Pi.

Terminal 1:

```
cd ~/ApexRover_Repo  
  
python3 dual_camera_server.py
```

Terminal 2:

```
cd ~/ApexRover_Repo  
  
python3 sensor_bridge.py
```

Then open the mobile app:

```
Connect  
Control  
Remote Control
```

You should see:

1. Camera video
 2. Sensor data at the top of the screen
 3. Manual control buttons
-

18. Summary

The correct order is:

1. Raspberry Pi connects to:

```
Apex_Rover_Net
```

1. Check ESP32:

```
curl http://192.168.4.1/get_status
```

1. Check cameras:

```
ls -l /dev/v4l/by-id/
```

1. Create:

```
dual_camera_server.py
```

1. Run:

```
python3 dual_camera_server.py
```

1. Test from the mobile browser:

```
http://192.168.4.2:5000/front_snapshot  
http://192.168.4.2:5000/arm_snapshot
```

1. Create:

```
sensor_bridge.py
```

1. Run:

```
python3 sensor_bridge.py
```

1. Test sensors:

```
curl "http://192.168.4.1/sensor_update?
pitch=2.4&roll=-1.1&front=35.6&rear=18.2&balance=STABLE"
```

1. Run the mobile app and open:

Remote Control

19. Important Notes

1. Raspberry Pi does not control robot movement.
2. Movement control is only from the mobile app.
3. Raspberry Pi is used only for cameras and sensors.
4. Mega sends `SENSOR:` lines to Raspberry Pi.
5. ESP32 receives `sensor_update` and broadcasts it to the mobile app.
6. If the Raspberry Pi IP changes, update the IP inside Flutter.
7. If sensor data does not appear, check:
 8. ESP32 code is uploaded and includes `/sensor_update`
 9. Mega code sends `SENSOR:` lines
 10. `MEGA_PORT` is correct
 11. Raspberry Pi is connected to `Apex_Rover_Net`
 12. Mobile app is connected to ESP32 WebSocket